

UML DIAGRAMS DOCUMENTATION

OF

TrustExam System - Online Examination Platform with Proctoring

A Comprehensive Analysis of System Architecture and Design

Date: April 2026

Table of Contents

1. Introduction
2. Use Case Specification
3. Use Case Diagram
4. Class Diagram
5. Functional and Non-Functional Requirements
6. Sequence Diagrams
7. Activity Diagrams
8. State Chart Diagrams
9. Component Diagram
10. Deployment Diagram
11. Database Schema
12. Architecture Diagram
13. Entity Relationship Diagram
14. Conclusion

1. Introduction

The TrustExam system is an advanced online examination platform designed to deliver secure, proctored examinations with intelligent trust scoring mechanisms. This UML documentation provides a comprehensive overview of the system's architecture, design patterns, and functional flows.

The TrustExam system incorporates:

- Multi-user role support (Students, Faculty, Administrators)
- Real-time behavioral monitoring and analysis
- Trust score calculation based on suspicious activities
- Secure examination session management
- Comprehensive result generation and reporting
- Activity logging and audit trails

This document presents all critical UML diagrams that define the system's structure, interactions, and behavior flows.

2. Use Case Specification

The Use Case model represents all interactions between system actors (Students, Faculty, and Administrators) and the various functionalities offered by the TrustExam system.

Key Actors:

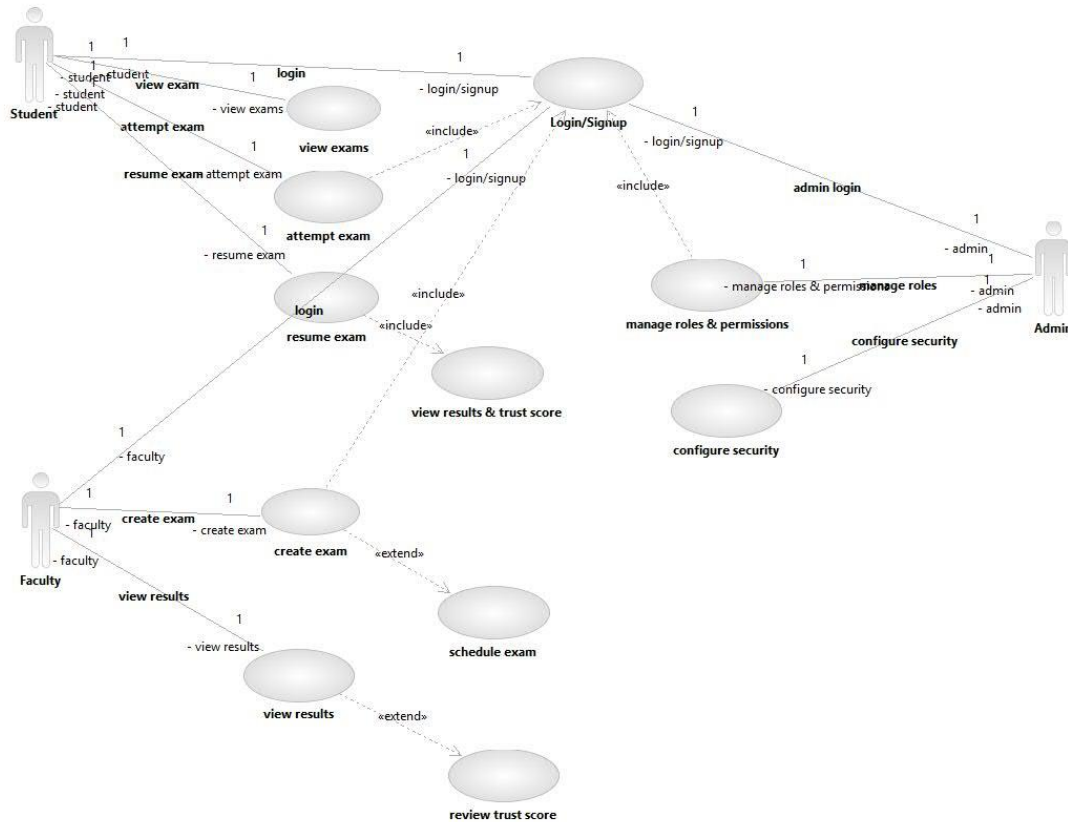
- Student: Takes examinations, views results, and participates in proctored sessions
- Faculty: Creates examinations, manages question banks, schedules exams, and reviews results
- Administrator: Manages system roles, permissions, and security configurations
- System: Manages authentication, trust scoring, and result generation

Core Use Cases

- Authentication & Access: Login/Signup - Multi-factor authentication and role-based access control
- Exam Management: Create Exam, Schedule Exam, View Exams - Faculty-initiated examination setup
- Examination: Attempt Exam, Resume Exam, Answer Questions - Core student examination process
- Proctoring: Capture Behavior Event, Calculate Trust Score - Real-time monitoring and analysis
- Results Management: View Results, Generate Results, Trust Score Calculation - Post-exam processing
- Security Management: Manage Roles & Permissions, Configure Security - Administrative controls

3. Use Case Diagram

The Use Case Diagram illustrates the interactions between system actors and the major use cases.



Description:

The diagram depicts three primary actors:

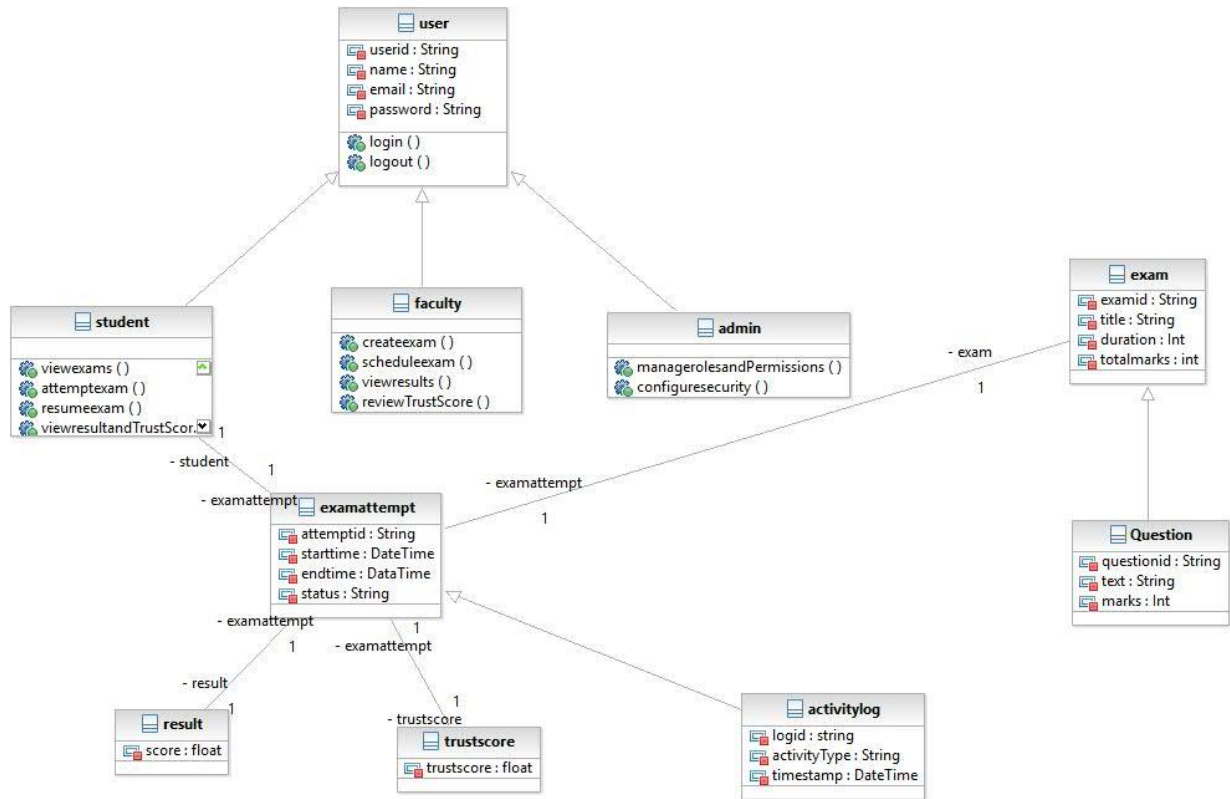
1. Student Actor: Engages in authentication, exam attempts, answer submission, and result viewing
2. Faculty Actor: Creates exams, schedules presentations, manages question banks, and monitors results
3. Administrator Actor: Manages roles, permissions, and security settings

Key workflows include:

- Login/Signup with role-based routing
- Exam creation with question integration (Create Exam → Schedule Exam)
- Exam participation with real-time monitoring (Attempt Exam → Resume Exam)
- Result generation with trust score integration
- Security configuration by administrators

4. Class Diagram

The Class Diagram represents the static structure of the TrustExam system, including all entities and their relationships.



Description:

The class diagram shows the following key entities:

1. User (Base Class):

- Attributes: `userId`, `name`, `email`, `password`
- Methods: `login()`, `logout()`

2. Student (Inherits from User):

- Methods: `viewExams()`, `attemptExam()`, `resumeExam()`, `viewResultsAndTrustScore()`

3. Faculty (Inherits from User):

- Methods: `createExam()`, `scheduleExam()`, `viewResults()`, `reviewTrustScore()`

4. Admin (Inherits from User):

- Methods: `manageRolesAndPermissions()`, `configureSecurity()`

5. ExamAttempt:

- Attributes: `attemptId`, `startTime`, `endTime`, `status`

- Associations: Links a Student to multiple attempts

6. Exam:

- Attributes: examId, title, duration, totalMarks
- Methods: Manages question integration and scheduling

7. Question:

- Attributes: questionId, text, marks
- Contains examination content

8. Result:

- Attributes: score, trustScore
- Calculates final outcomes with integrity measures

9. ActivityLog:

- Tracks user behaviors and enables audit trails

10. TrustScore:

- Calculates and stores behavioral assessment metrics
- Identifies suspicious activities

Relationships:

- One-to-Many: Student to ExamAttempt, Faculty to Exam
- Inheritance hierarchies for User roles
- Composition and dependencies between business entities

5. Functional and Non-Functional Requirements

Status	ID	Risk	Source ID	Name	Description	Owner
☐	68	○○○○○		Timed Examination with Auto-Submission	The system shall support time-limited exams where students m...	Unassigned
☐	69	○○○○○		Answer Auto-Saving and Session Monitoring	The system shall continuously auto-save student answers durin...	Unassigned
☐	65	○○○○○		User Authentication and Role Management	The system shall provide secure login and authentication for th...	Unassigned
☐	67	○○○○○		Randomized Question Delivery	The system shall present questions in a randomized order for e...	Unassigned
☐	75	○○○○○		Performance	The system shall provide fast response times and low latency d...	Unassigned
☐	70	○○○○○		Result Generation with Trust Score	The system shall generate exam results by combining student ...	Unassigned
☐	66	○○○○○		Exam Creation and Management	The system shall allow faculty members to create, edit, and m...	Unassigned
☐	74	○○○○○		Maintainability	The system shall be developed using a modular and well-struct...	Unassigned
☐	77	○○○○○		Auditability	The system shall maintain secure and immutable activity logs...	Unassigned
☐	71	○○○○○		Security	The system shall ensure secure access and data protection by u...	Unassigned
☐	72	○○○○○		Reliability	The system shall operate consistently and reliably, even in the p...	Unassigned
☐	76	○○○○○		Privacy	The system shall respect user privacy by not requiring mandato...	Unassigned
☐	73	○○○○○		Scalability	The system shall be capable of handling multiple users and ex...	Unassigned

Description:

The system is designed with comprehensive functional and non-functional requirements:

Functional Requirements:

- FR1: Timed Examination with Auto-Submission - Support time-limited exams with automatic submission
- FR2: Answer Auto-Saving and Session Monitoring - Continuous monitoring of answer entries
- FR3: User Authentication and Role Management - Secure login with role-based access
- FR4: Randomized Question Delivery - Questions presented in random order for fairness
- FR5: Real-time Performance Evaluation - Immediate result calculation combining marks and trust scores
- FR6: Exam Creation and Management - Faculty can create, edit, and manage examinations
- FR7: Result Generation with Trust Score - Comprehensive result reports with behavioral analysis

Non-Functional Requirements:

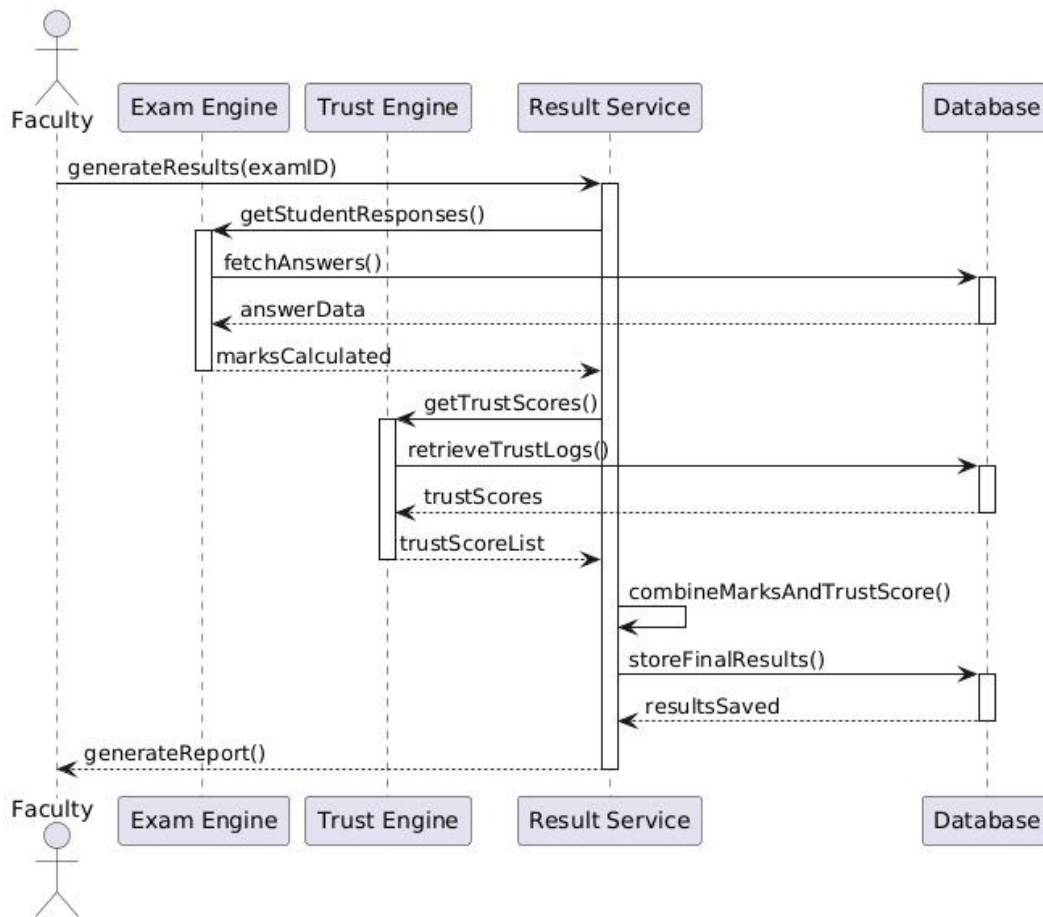
- NF75: Performance - System shall provide fast response times and low latency
- NF70: Result Generation with Trust Score - Concurrent user handling capability
- NF66: Exam Creation and Management - Robust management of exam metadata
- NF74: Maintainability - Modular and well-structured codebase for easy maintenance
- NF77: Auditability - Comprehensive logging of all user activities
- NF71: Security - Secure access control and data protection mechanisms
- NF72: Reliability - System consistency and reliable operation
- NF76: Privacy - User privacy protection and data confidentiality

- NF73: Scalability - System capability to handle growing user base and data

6. Sequence Diagrams

Sequence Diagrams depict the interactions between system components and actors over time.

6.1 Result Generation Sequence



Description - Result Generation Process:

This sequence demonstrates how examination results are generated and combined with trust scores:

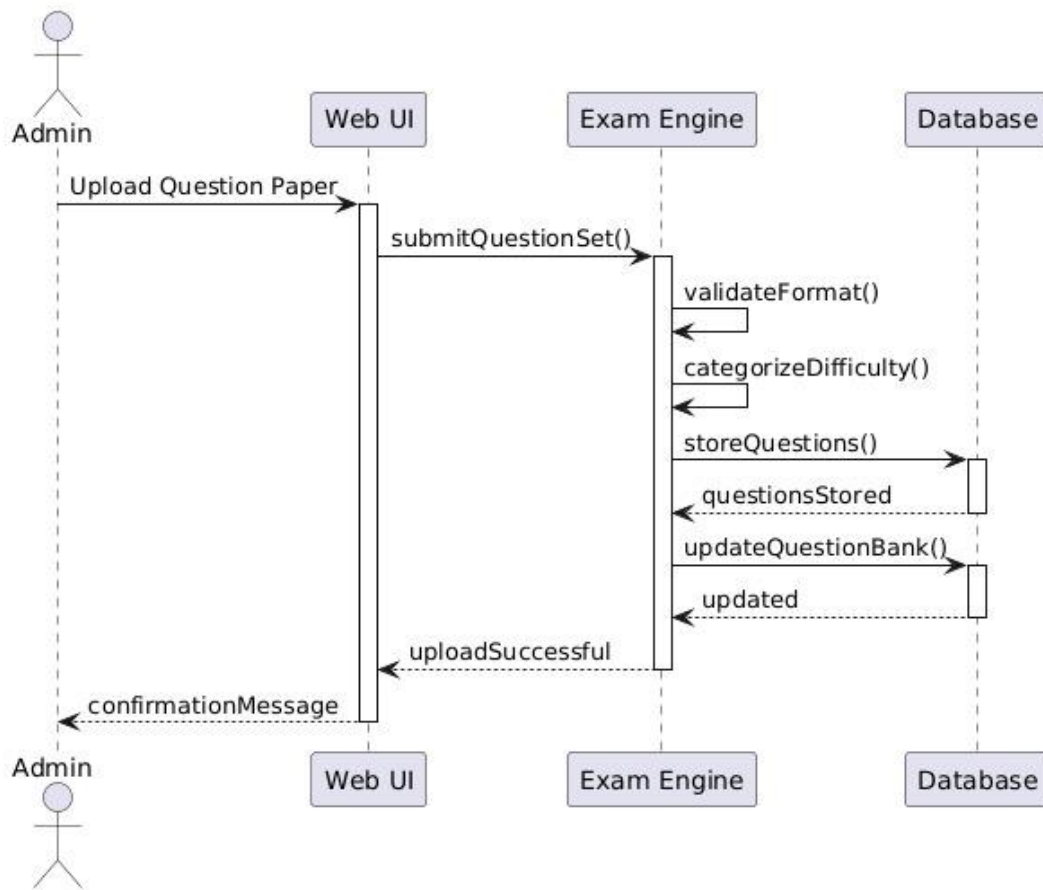
1. Faculty initiates result generation request for an exam
2. Exam Engine retrieves student responses from the database
3. Exam Engine processes answers and fetches correct answers
4. Marks are calculated based on student responses
5. Trust Engine retrieves trust logs and calculates trust scores
6. Result Service combines marks and trust scores
7. Final results are stored in the database
8. Faculty receives the comprehensive report

Key Participants:

- Faculty: Initiator of result generation
- Exam Engine: Processes answers and calculates marks
- Trust Engine: Evaluates behavioral data and trust scores
- Result Service: Combines data and generates final reports

- Database: Persistent storage of results and related data

6.2 Question Upload Sequence



Description - Question Upload Process:

This sequence shows how administrators upload and validate examination questions:

1. Admin uploads a question paper file through the Web UI
2. Exam Engine validates the file format and structure
3. Questions are categorized by difficulty level
4. Questions are stored in the database
5. Question bank is updated with new entries
6. Confirmation message is sent back to admin

Key Participants:

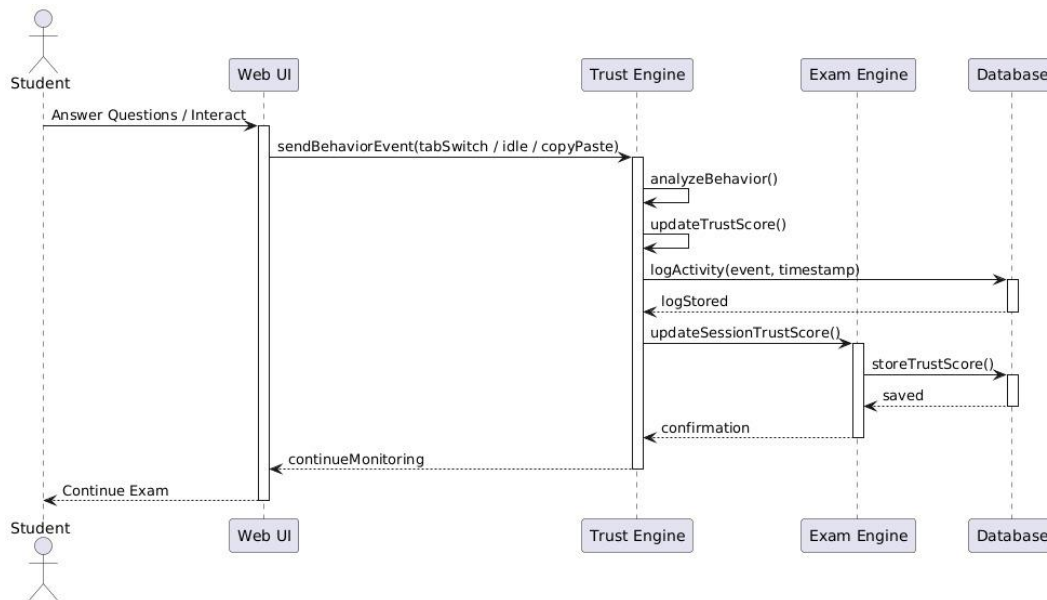
- Admin: Uploads question files
- Web UI: User interface for file upload
- Exam Engine: Validates and processes questions
- Database: Stores question bank data

Validation Steps:

- Format validation (supported file types)
- Content validation (question structure)

- Metadata validation (question attributes)

6.3 Student Exam Interaction Sequence (Behavioral Monitoring)



Description - Student Behavior Monitoring:

This sequence depicts real-time behavior analysis during student examination:

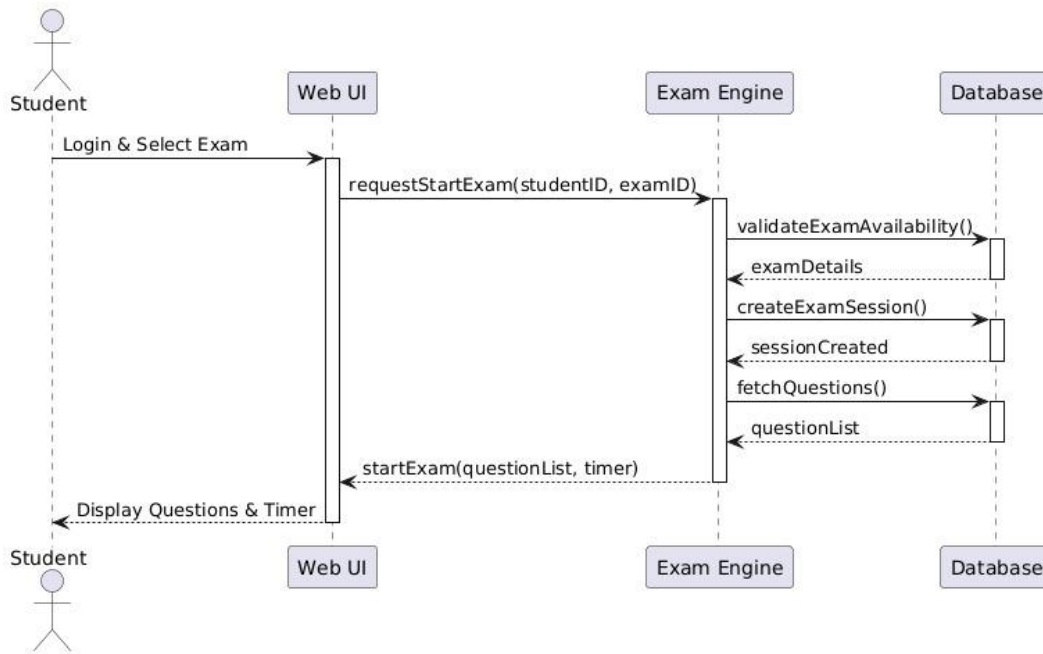
1. Student interacts with questions (tab switches, copy-paste, idle time)
2. Web UI captures behavior events
3. Trust Engine analyzes behavior patterns for suspicious activity
4. If suspicious: Trust score is reduced
5. Trust Engine logs the activity with timestamps
6. Session trust score is updated
7. Exam Engine continues to monitor

Behavioral Indicators Tracked:

- Tab switching (indicates leaving the exam window)
- Copy-paste operations (potential content sharing)
- Excessive idle time (question disengagement)
- Unusual interaction patterns (specific response speeds)
- Mouse movement patterns
- Keyboard activity analysis

The system maintains continuous monitoring and adjusts trust scores dynamically based on detected behaviors.

6.4 Student Login and Exam Selection Sequence



Description - Login and Exam Session Initialization:

This sequence shows the complete flow from student login to exam session creation:

1. Student enters credentials
2. Web UI validates and submits login request
3. Exam Engine validates credentials and checks availability
4. Student retrieves available exams
5. Student selects an examination
6. Exam Engine validates exam availability and student eligibility
7. Exam session is created in the database
8. Question paper is fetched and loaded
9. Timer is initialized
10. Questions are displayed to the student

Key Components:

- Web UI: Login interface and exam selection
- Exam Engine: Authentication and authorization
- Database: Stores exam metadata and student information

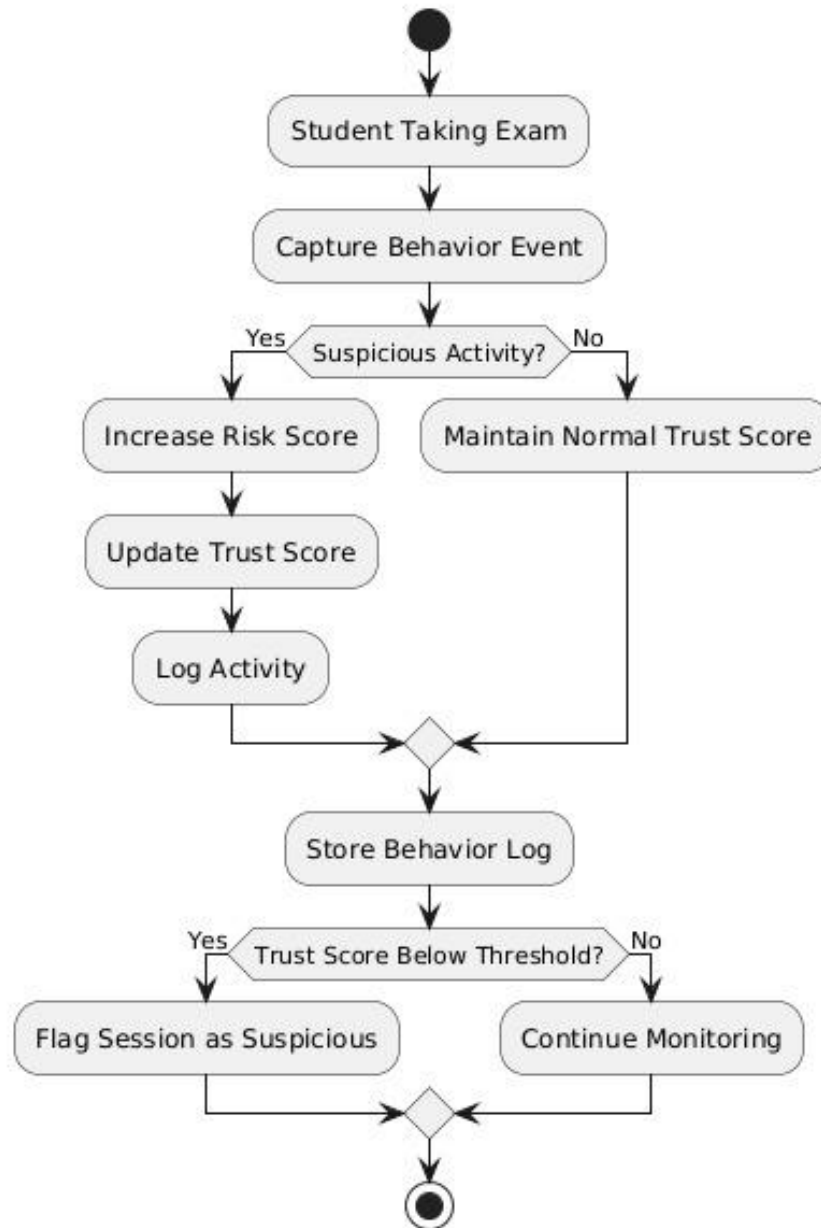
Session Management:

- Session creation with unique identifiers
- Timer initialization with duration validation
- Question randomization and delivery
- Real-time session tracking for resumption capability

7. Activity Diagrams

Activity Diagrams represent the workflow and flow of control within specific processes. They show the logical sequence of activities and decision points in the system.

7.1 Student Taking Exam Activity Diagram



Description - Student Exam Flow:

The diagram illustrates the complete workflow of a student taking an examination with behavioral monitoring:

Flow Steps:

1. Student Taking Exam (Initial state)
2. System captures behavior events
3. Decision: Is the activity suspicious?
 - YES: Increase Risk Score → Update Trust Score → Log Activity

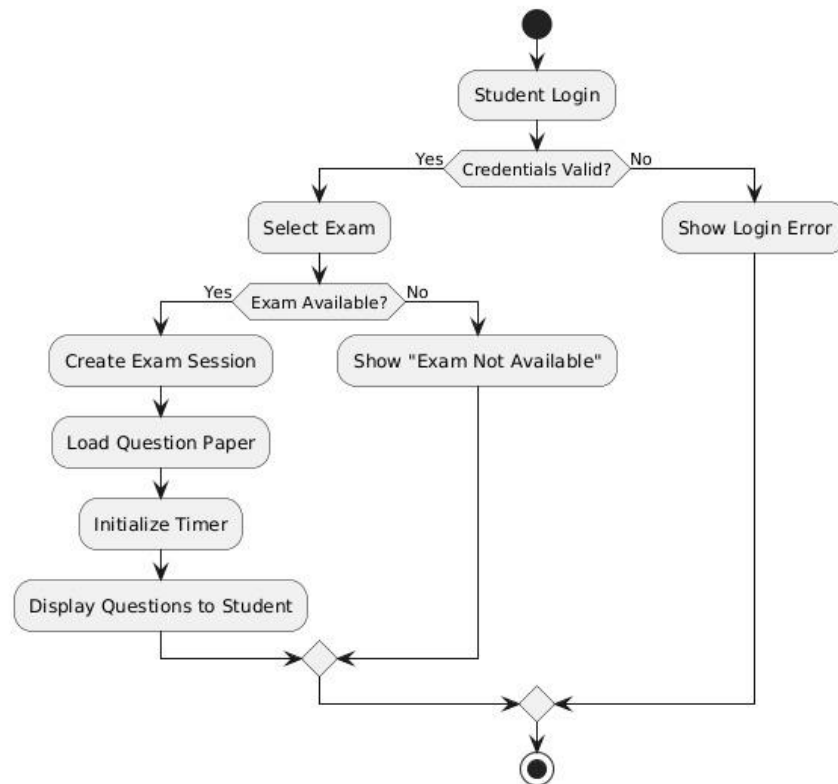
- NO: Maintain Normal Trust Score
- 4. Store Behavior Log
- 5. Decision: Is Trust Score below threshold?
 - YES: Flag Session as Suspicious
 - NO: Continue monitoring
- 6. End state

Key Decision Points:

- Suspicious Activity Detection: Identifies unusual behavioral patterns
- Score Threshold Evaluation: Determines if session should be flagged
- Continuous Monitoring: Real-time analysis throughout the exam

This activity ensures real-time proctoring and maintains exam integrity.

7.2 Student Login Activity Diagram



Description - Student Login Process:

The diagram shows the authentication and exam access workflow:

Flow Steps:

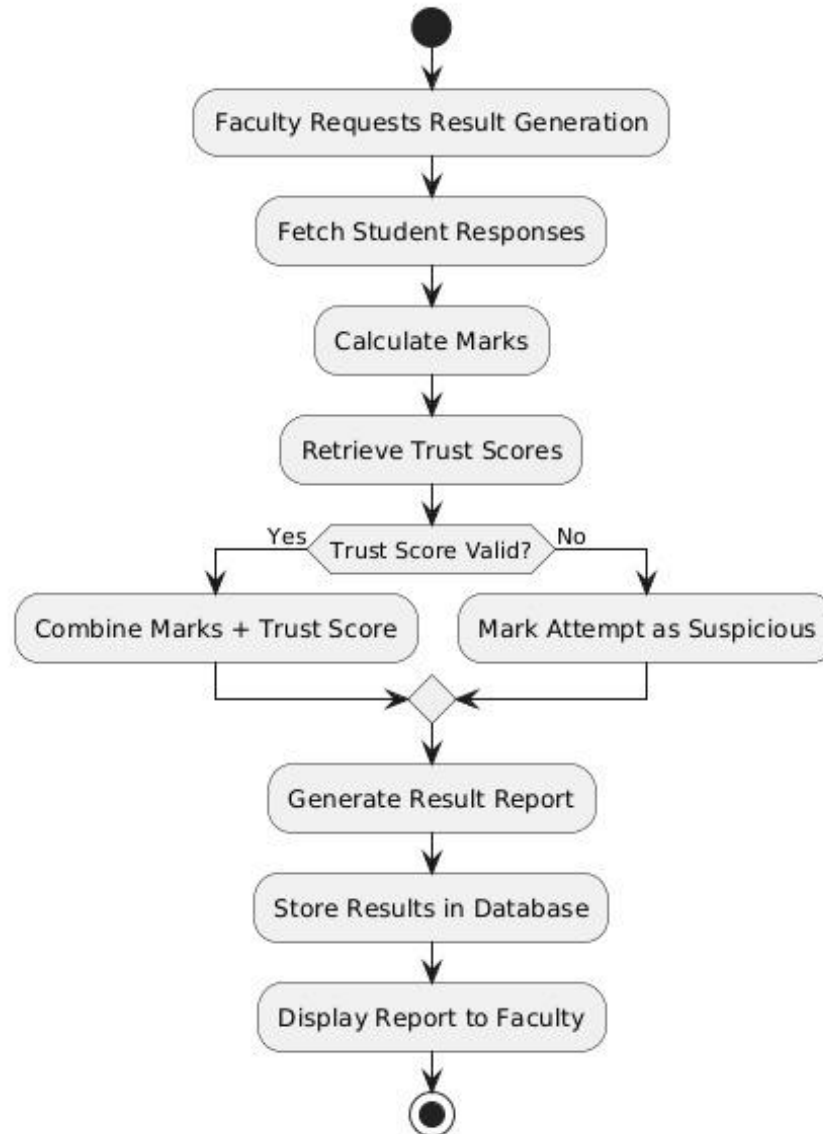
1. Student Login (Initial state)
2. Decision: Are credentials valid?
 - NO: Show login error and return to login
 - YES: Proceed to exam selection
3. Select Exam
4. Decision: Is exam available?
 - NO: Show "Exam Not Available" message
 - YES: Proceed
5. Create Exam Session
6. Load Question Paper
7. Initialize Timer
8. Display Questions to Student (Final state)

Key Decision Points:

- Credential Validation: Ensures authorized access
- Exam Availability Check: Verifies exam scheduling and eligibility
- Session Creation: Establishes unique exam instance

This workflow ensures secure access and proper session management.

7.3 Faculty Result Generation Activity Diagram



Description - Result Generation and Evaluation Process:

The diagram represents the complete result generation workflow:

Flow Steps:

1. Faculty Requests Result Generation (Initial state)
2. Fetch Student Responses
3. Calculate Marks for each response
4. Retrieve Trust Scores from trust engine
5. Decision: Are trust scores valid?
 - YES: Combine Marks + Trust Score
 - NO: Mark attempt as suspicious
6. Generate Result Report

7. Store Results in Database

8. Display Report to Faculty (Final state)

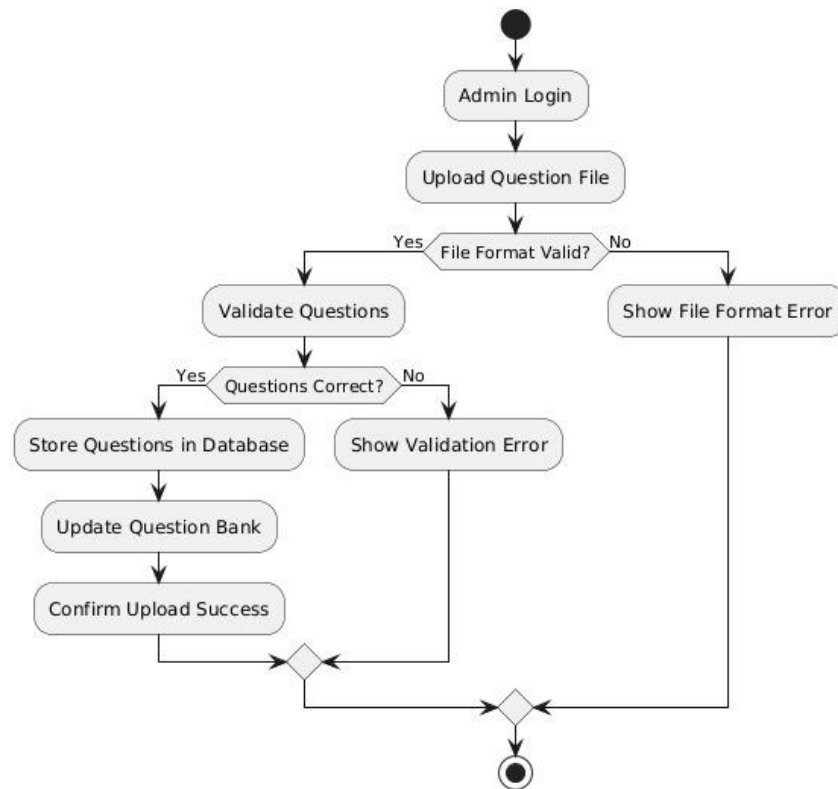
Key Components:

- Answer Evaluation: Automatic marking based on correct answers
- Trust Score Integration: Incorporates behavioral analysis
- Result Validation: Ensures data integrity
- Report Generation: Creates comprehensive assessment documents

Decision Points:

- Trust Score Validity: Checks behavioral data completeness
- Score Threshold Verification: Identifies questionable attempts

7.4 Admin Question Upload Activity Diagram



Description - Question Upload and Validation:

The diagram depicts the administrative question management process:

Flow Steps:

1. Admin Login (Initial state)
2. Upload Question File
3. Validate File Format
 - Decision: Is format valid?
 - NO: Show File Format Error → Return to upload
4. Validate Questions Content
 - Decision: Are all questions correct?
 - NO: Show Validation Error → Return to upload
5. Store Questions in Database
6. Update Question Bank
7. Confirm Upload Success (Final state)

Validation Stages:

- File Format Validation: Checks supported file types (CSV, XML, PDF)
- Content Structure Validation: Verifies question metadata
- Question Correctness: Ensures all required fields are present

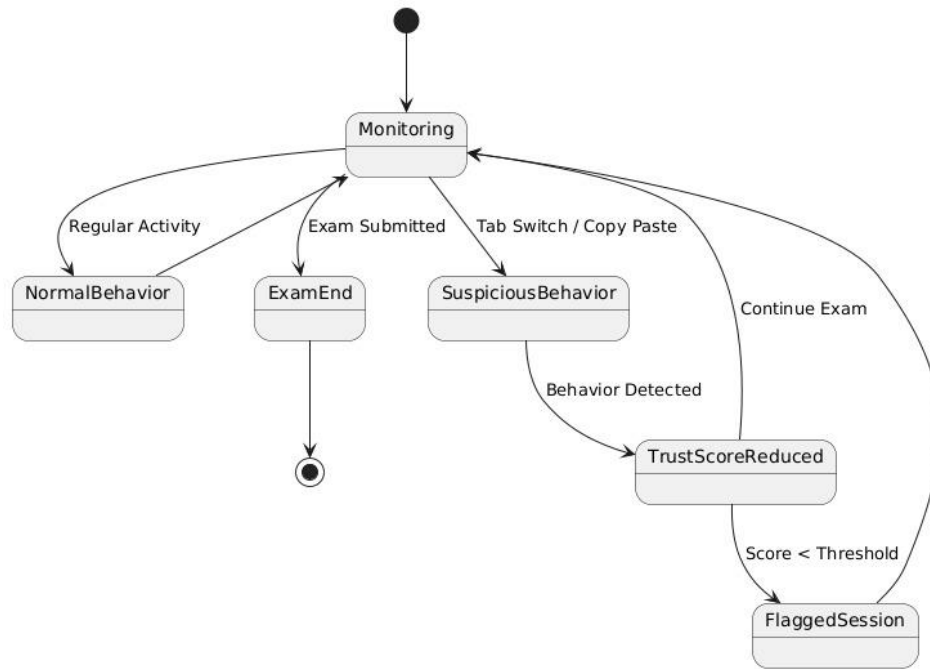
Error Handling:

- Format errors are immediately reported with correction guidance
- Content errors are detailed for specific question items
- Failed uploads allow for file correction and resubmission

8. State Chart Diagrams

State Chart Diagrams represent the different states an entity can be in and the transitions between those states based on events or conditions.

8.1 Exam Monitoring State Diagram



Description - Behavioral Monitoring States:

This state diagram shows the behavioral monitoring lifecycle during examination:

States:

1. **Monitoring (Initial State):**
 - System actively monitors student behavior
 - Listens for behavior events
2. **NormalBehavior:**
 - State triggered by: Regular Activity event
 - Student demonstrates normal exam behavior
 - Previous history is preserved
3. **ExamEnd:**
 - State triggered by: Exam Submitted event
 - Marks the conclusion of examination session
 - Final state
4. **SuspiciousBehavior:**
 - State triggered by: Tab Switch / Copy Paste events
 - Indicates detection of unusual activity
 - Transitions to TrustScoreReduced

5. TrustScoreReduced:

- State entered after suspicious activity
- Trust score is decremented
- Can transition to FlaggedSession if score falls below threshold

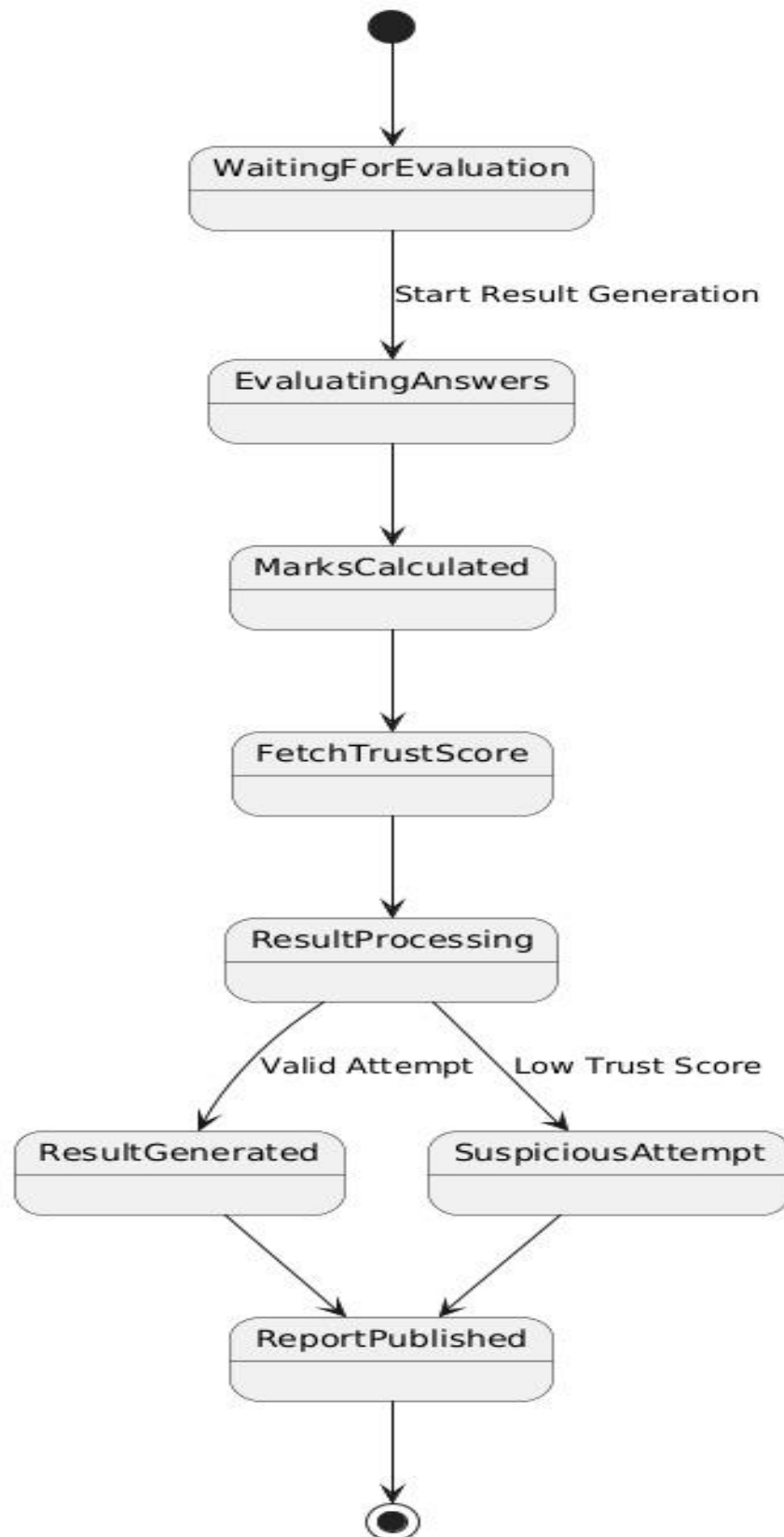
6. FlaggedSession:

- State triggered by: Score < Threshold
- Session marked for manual review
- Exam may be flagged as compromised

Transitions:

- Monitoring → NormalBehavior: Regular user activity detected
- Monitoring → SuspiciousBehavior: Suspicious activity (tab switch, copy-paste) detected
- SuspiciousBehavior → TrustScoreReduced: Trust score adjustment
- TrustScoreReduced → FlaggedSession: Score drops below threshold
- Any State → ExamEnd: Exam time expires or student submits
- TrustScoreReduced → Continue Exam: If score remains above threshold

8.2 Result Generation State Diagram



Description - Result Processing States:

This state diagram illustrates the result generation and publication workflow:

States:

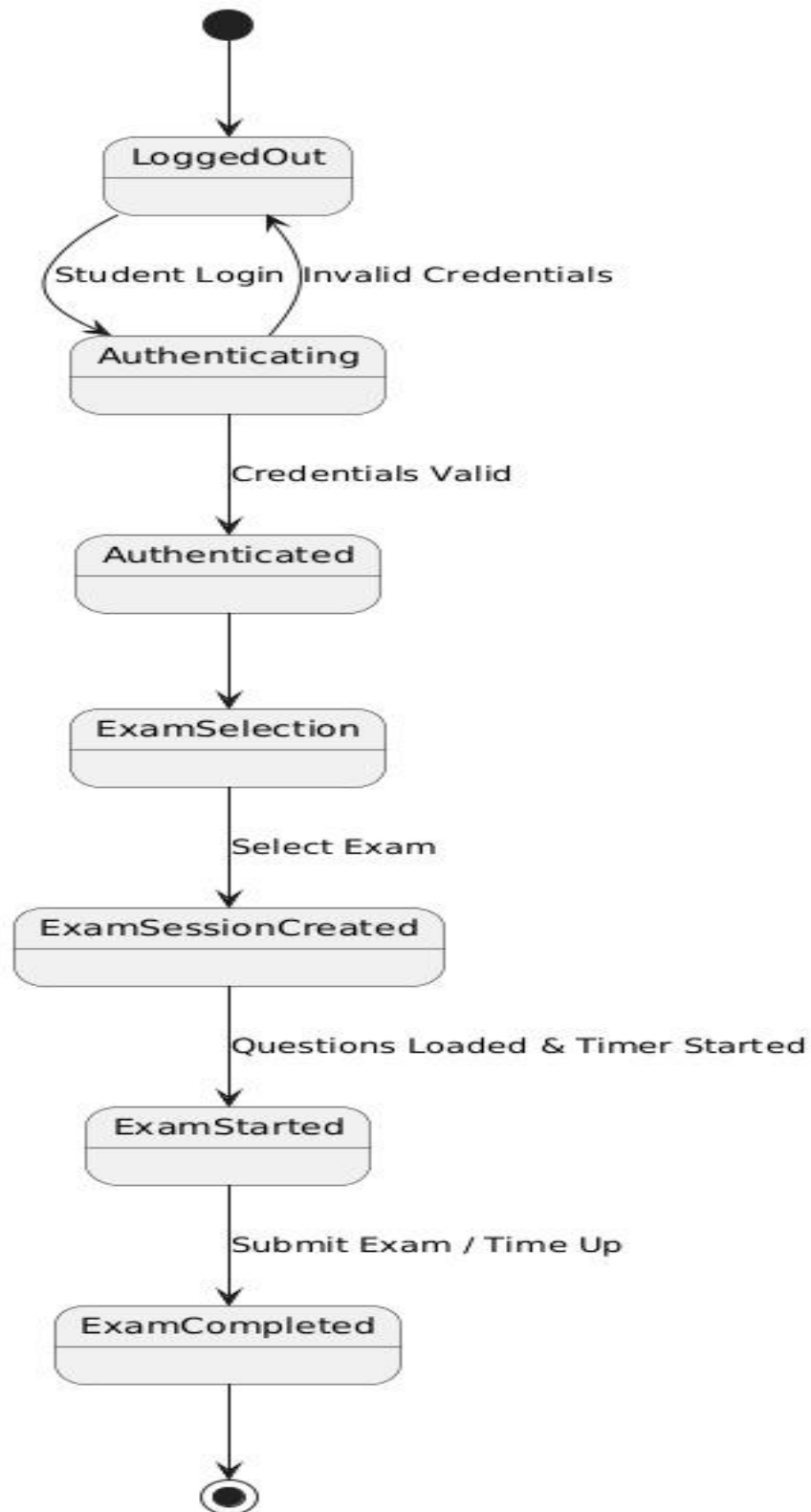
1. WaitingForEvaluation (Initial):
 - Student has completed the exam
 - System waiting for result generation trigger
2. Start Result Generation:
 - Trigger by: Faculty request or scheduled process
 - Begin answer evaluation
3. EvaluatingAnswers:
 - Processing student responses
 - Matching against answer key
 - Calculating marks for each question
4. MarksCalculated:
 - All answers have been evaluated
 - Total marks computed
 - Ready for trust score integration
5. FetchTrustScore:
 - Retrieve behavioral analysis data
 - Get trust score value from system
6. ResultProcessing:
 - Combine marks with trust score
 - Apply weightage algorithms
 - Prepare final result
7. ResultGenerated:
 - Result successfully created with valid attempt
 - Result ready for publication
8. SuspiciousAttempt:
 - State triggered by: Low Trust Score
 - Result marked as potentially compromised
 - Flags attempt for manual review
9. ReportPublished:
 - Final state

- Result made available to student and faculty

Decision Point:

- Valid Attempt: Trust score above threshold → ResultGenerated
- Low Trust Score: Suspicious activity detected → SuspiciousAttempt

8.3 Student Login State Diagram



Description - Authentication and Session Initialization:

This state diagram represents the student login and exam access flow:

States:

1. LoggedOut (Initial):

- Student not authenticated
- Access to exam system blocked

2. Authenticating:

- State entered when: Student submits credentials
- System verifies username and password
- Checks authentication database

3. Authenticated:

- State triggered by: Valid Credentials
- Student identity verified
- Ready to access exam functions

4. ExamSelection:

- State entered after: Successful authentication
- Student views available exams
- Selects desired examination

5. ExamSessionCreated:

- State triggered by: Select Exam
- System creates unique exam session
- Associates session with student

6. ExamStarted:

- State entered when: Questions Loaded & Timer Started
- Student begins answering questions
- Real-time monitoring activated

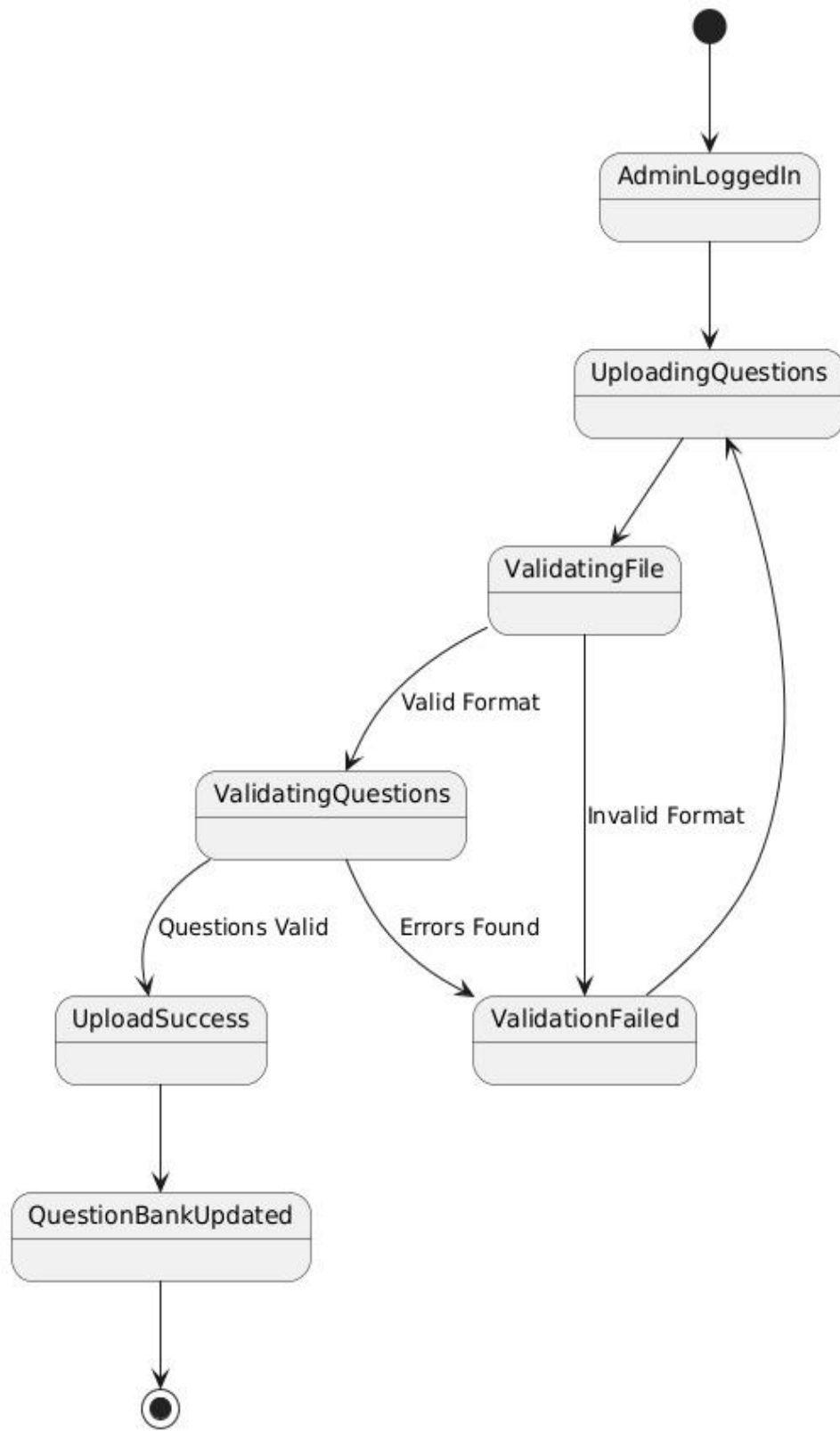
Transitions:

- LoggedOut → Authenticating: Login attempt
- Authenticating → Authenticated: Valid credentials (Credentials Valid)
- Authenticating → LoggedOut: Invalid credentials (Invalid Credentials)
- Authenticated → ExamSelection: System displays available exams
- ExamSelection → ExamSessionCreated: Student selects exam
- ExamSessionCreated → ExamStarted: Questions loaded and timer initiated
- ExamStarted → ExamCompleted: Submit or time expires (Submit Exam/Time Up)

Key Points:

- Multi-step authentication process
- Session creation before exam access
- Timer initialization at exam start
- Question randomization during session creation

8.4 Admin Question Upload State Diagram



Description - Question Management Lifecycle:

This state diagram shows the admin question upload and validation process:

States:

1. AdminLoggedIn (Initial):

- Admin successfully authenticated
- Ready to perform administrative tasks

2. UploadingQuestions:

- State entered when: Admin selects file upload
- File transmission in progress
- System receives file data

3. ValidatingFile:

- State triggered by: File upload complete
- System checks file format
- Verifies file structure and integrity

4. ValidatingQuestions:

- State entered when: File format is valid
- Individual questions are validated
- Metadata and content verification

5. UploadSuccess:

- State triggered by: All questions valid
- Questions stored in database
- Question bank updated

6. ValidationFailed:

- State entered when: Format or content errors detected
- Error details provided to admin
- Allows for file correction and resubmission

7. QuestionBankUpdated:

- Final state
- Questions now available for exam creation

Transitions:

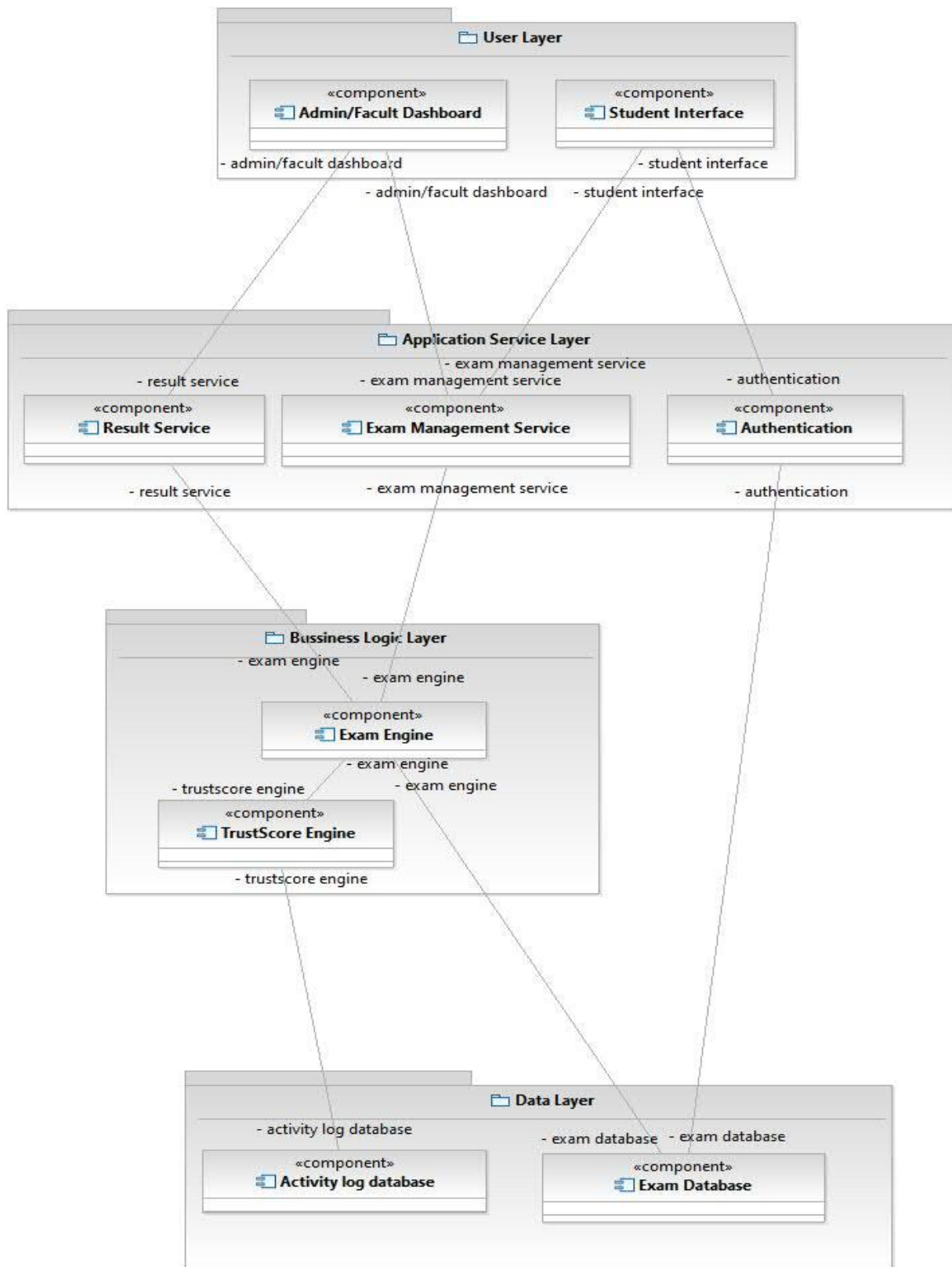
- AdminLoggedIn → UploadingQuestions: Initiate upload
- UploadingQuestions → ValidatingFile: File received
- ValidatingFile → ValidatingQuestions: Valid format (Valid Format)
- ValidatingFile → ValidationFailed: Invalid format (Invalid Format)

- ValidatingQuestions → UploadSuccess: All questions valid (Questions Valid)
- ValidatingQuestions → ValidationFailed: Errors found (Errors Found)
- ValidationFailed → UploadingQuestions: Retry upload
- UploadSuccess → QuestionBankUpdated: Finalize

Key Features:

- Multi-stage validation ensures data quality
- Error feedback allows corrections
- Atomic updates to question bank
- Audit trail for administrative actions

9. Component Diagram



Description - System Architecture Overview:

The Component Diagram illustrates the layered architecture of the TrustExam system:

User Layer:

- Admin/Faculty Dashboard: Management interface for administrative users
- Student Interface: Examination and result viewing interface for students
- Responsibilities: User interaction, interface rendering, user input handling

Application Service Layer:

- Result Service: Processes and aggregates results with trust scores
- Exam Management Service: Handles exam lifecycle management
- Authentication Service: Manages user authentication and authorization
- Responsibilities: Business logic, service orchestration, cross-component coordination

Business Logic Layer:

- Exam Engine: Core examination processing and question management
- Trust Score Engine: Behavioral analysis and trust calculation
- Responsibilities: Complex computations, rule enforcement, core algorithms

Data Layer:

- Activity Log Database: Stores all user activity and behavior logs
- Exam Database: Maintains examination metadata, questions, and sessions
- Responsibilities: Data persistence, query processing, transaction management

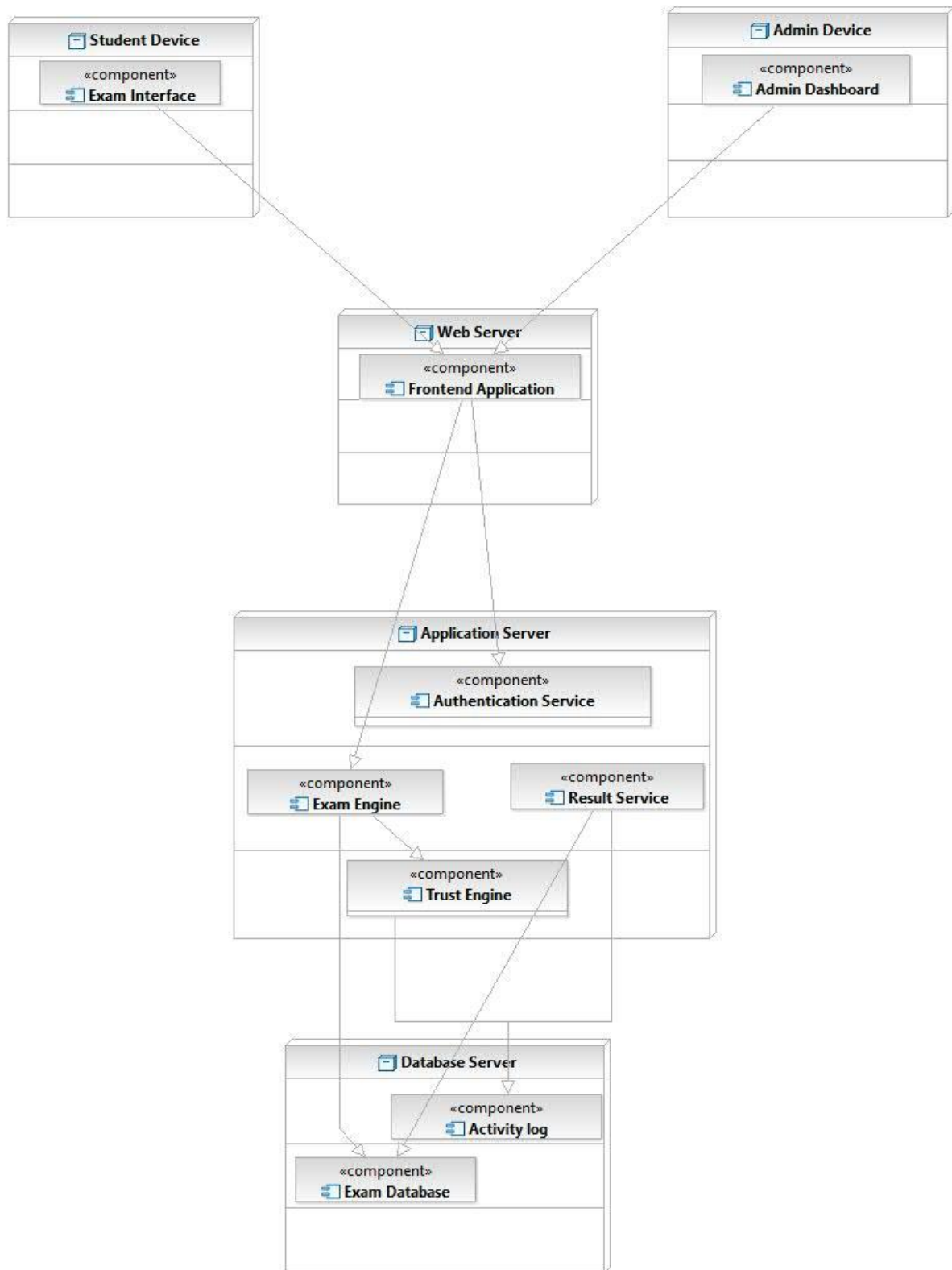
Inter-Layer Dependencies:

- User Layer communicates with Application Services
- Application Services orchestrate Business Logic components
- Business Logic accesses Data Layer for persistence
- Services are loosely coupled through well-defined interfaces

Benefits of Layered Architecture:

- Separation of concerns: Each layer has distinct responsibilities
- Reusability: Components can be reused across different services
- Scalability: Layers can be independently scaled based on demand
- Maintainability: Changes localized to specific layers
- Testability: Components can be tested in isolation

10. Deployment Diagram



Description - System Deployment Architecture:

The Deployment Diagram shows how system components are distributed across physical nodes:

Deployment Nodes:

1. Student Device:

- Component: Exam Interface
- Role: Student-facing web application
- Responsibilities: Question display, answer input, behavior capture
- Connection: HTTPS to Web Server
- Features: Responsive design, real-time event capture

2. Admin Device:

- Component: Admin Dashboard
- Role: Administrative management interface
- Responsibilities: System configuration, user management, report generation
- Connection: HTTPS to Web Server
- Security: Role-based access control

3. Web Server:

- Component: Frontend Application
- Role: HTTP/HTTPS request handling
- Responsibilities: Static content delivery, request routing, session management
- Protocol: HTTP/HTTPS
- Load Balancing: Distributes requests across servers

4. Application Server:

- Components:
 - Authentication Service: User credential verification
 - Exam Engine: Question delivery and answer processing
 - Trust Engine: Behavioral analysis
 - Result Service: Result compilation and reporting
- Role: Core business logic execution
- Responsibilities: Service orchestration, business rule enforcement
- Protocols: Internal APIs, service-to-service communication

5. Database Server:

- Components:
 - Activity Log Database: Behavior and audit logging
 - Exam Database: Questions, exams, sessions, and results
- Role: Persistent data storage
- Responsibilities: Data persistence, transaction management, query processing

- Access: Secured database connections from Application Server

Network Architecture:

- Client-Server model: Student and Admin devices communicate with Web Server
- Multi-layered: Application Server separates business logic from data layer
- Secure Channels: All communications encrypted with HTTPS/TLS
- Database Isolation: Separate databases for activity logging and exam data

Communication Flows:

1. Student login → Web Server → Application Server → Authentication Service
2. Question retrieval → Web Server → Application Server → Exam Engine → Database
3. Behavior monitoring → Web Server → Application Server → Trust Engine → Activity Log
4. Result generation → Application Server → Result Service → Exam Database
5. Admin queries → Web Server → Application Server → Analytics Service → Databases

Scalability Considerations:

- Horizontal scaling: Multiple Web Server instances behind load balancer
- Application Server clustering: Distributed processing across multiple nodes
- Database replication: Read replicas for query distribution
- Caching layer: Redis/Memcached for session and frequently accessed data
- CDN integration: Content delivery network for static assets

Security Features:

- SSL/TLS encryption for all network communications
- Database access restricted to Application Server
- Network segmentation: Firewalls between layers
- Authentication and authorization checks at entry points
- Activity logging for audit trails and forensic analysis

11. Database Schema

The database schema defines the structure and relationships of all tables in the TrustExam system.

It provides a persistent storage layer for all system data including users, exams, questions, answers, and results.

Database Tables and Structure:

1. USER Table

- Purpose: Stores login credentials and role information for all system users
- Columns:
 - user_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique user identifier
 - name (VARCHAR(100), NOT NULL): User's full name
 - email (VARCHAR(100), UNIQUE, NOT NULL): User's email address
 - password_hash (VARCHAR(255), NOT NULL): Encrypted password
 - role (VARCHAR(20), NOT NULL): User role (Student/Faculty/Admin)

2. EXAM Table

- Purpose: Stores examination metadata and configuration
- Columns:
 - exam_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique exam identifier
 - title (VARCHAR(100), NOT NULL): Examination title
 - duration (INT, NOT NULL): Exam duration in minutes
 - created_by (INT, FOREIGN KEY): Reference to user who created the exam

3. QUESTION Table

- Purpose: Stores individual questions for each exam
- Columns:
 - question_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique question identifier
 - exam_id (INT, FOREIGN KEY): Reference to parent exam
 - question_text (TEXT, NOT NULL): Full question text
 - marks (FLOAT, NOT NULL): Marks allocated for this question

4. OPTION_TABLE

- Purpose: Stores multiple choice options for each question
- Columns:
 - option_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique option identifier
 - question_id (INT, FOREIGN KEY): Reference to parent question
 - option_text (VARCHAR(255), NOT NULL): Text of the option
 - is_correct (BOOLEAN, DEFAULT FALSE): Indicates if option is correct

5. EXAM_ATTEMPT Table

- Purpose: Tracks each student's examination session

- Columns:

- attempt_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique attempt identifier
- exam_id (INT, FOREIGN KEY): Reference to exam being attempted
- user_id (INT, FOREIGN KEY): Reference to student taking exam
- start_time (DATETIME): When exam session started
- submit_time (DATETIME): When exam was submitted
- trust_score (FLOAT): Behavioral trust score for this attempt

6. ANSWER Table

- Purpose: Stores student responses for each question

- Columns:

- answer_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique answer identifier
- attempt_id (INT, FOREIGN KEY): Reference to exam attempt
- question_id (INT, FOREIGN KEY): Reference to question
- selected_option_id (INT, FOREIGN KEY): Reference to selected option
- obtained_marks (FLOAT): Marks awarded for this answer

7. RESULT Table

- Purpose: Stores final examination results and scores

- Columns:

- result_id (INT, PRIMARY KEY, AUTO_INCREMENT): Unique result identifier
- attempt_id (INT, UNIQUE, FOREIGN KEY): Reference to exam attempt
- total_score (FLOAT): Final score combining marks and trust score
- final_status (VARCHAR(20)): Pass/Fail/Review status

Key Relationships:

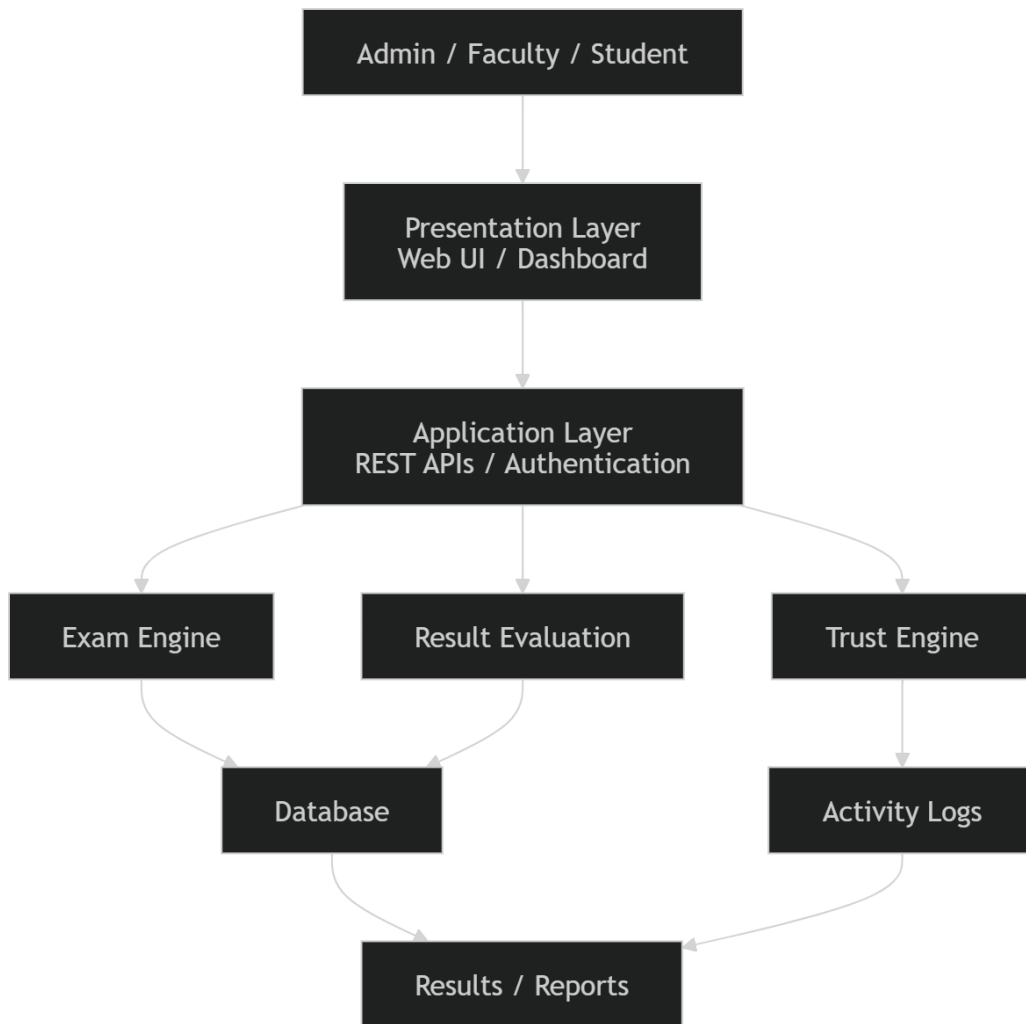
- USER → EXAM (1:Many): One user can create multiple exams
- USER → EXAM_ATTEMPT (1:Many): One student can attempt multiple exams
- EXAM → QUESTION (1:Many): One exam contains multiple questions
- QUESTION → OPTION_TABLE (1:Many): One question has multiple options
- EXAM_ATTEMPT → ANSWER (1:Many): One attempt contains multiple answers
- EXAM_ATTEMPT → RESULT (1:1): One attempt produces one final result

Database Design Principles:

- Normalization: Tables are normalized to eliminate data redundancy
- Referential Integrity: Foreign key constraints maintain data consistency
- Scalability: Schema supports growing number of users and exams
- Auditability: Timestamps enable tracking of exam sessions
- Data Isolation: Activity logs separate from exam data for performance

12. Architecture Diagram

The Architecture Diagram illustrates the high-level system structure and how different layers communicate with each other.



Description - System Architecture Layers:

1. Presentation Layer

- Components: Web UI / Dashboard
- Users: Admin, Faculty, Student
- Responsibilities:
 - User interface rendering
 - Input validation and formatting
 - User session management
- Technologies: HTML5, CSS3, JavaScript/React/Vue.js
- Features: Responsive design, real-time updates

2. Application Layer

- Components: REST APIs, Authentication Service
- Responsibilities:
 - Request routing and handling
 - Authentication and authorization
 - API endpoint management
 - Cross-layer communication
- Technologies: Node.js/Express, Spring Boot, Django
- Security: JWT tokens, OAuth2, role-based access control

3. Business Logic Layer

- Components:
 - Exam Engine: Manages exam creation, delivery, and question randomization
 - Result Evaluation: Calculates marks and processes answers
 - Trust Engine: Analyzes student behavior and computes trust scores
- Responsibilities:
 - Core business rule enforcement
 - Complex calculations and algorithms
 - Data transformation and processing
- Technologies: Custom business logic, rule engines

4. Data Layer

- Components: Database, Activity Logs
- Responsibilities:
 - Persistent data storage
 - Query processing and indexing
 - Transaction management
 - Data backup and recovery
- Technologies: MySQL/PostgreSQL, Redis Cache
- Storage:
 - Exam Database: Exam metadata, questions, answers, results
 - Activity Logs: User behavior, system events, audit trails

5. Output Layer

- Components: Results/Reports
- Responsibilities:
 - Result compilation and formatting
 - Report generation
 - Export functionality (PDF, Excel)

Architecture Communication Flow:

1. User Interface → Application Layer: User actions and requests

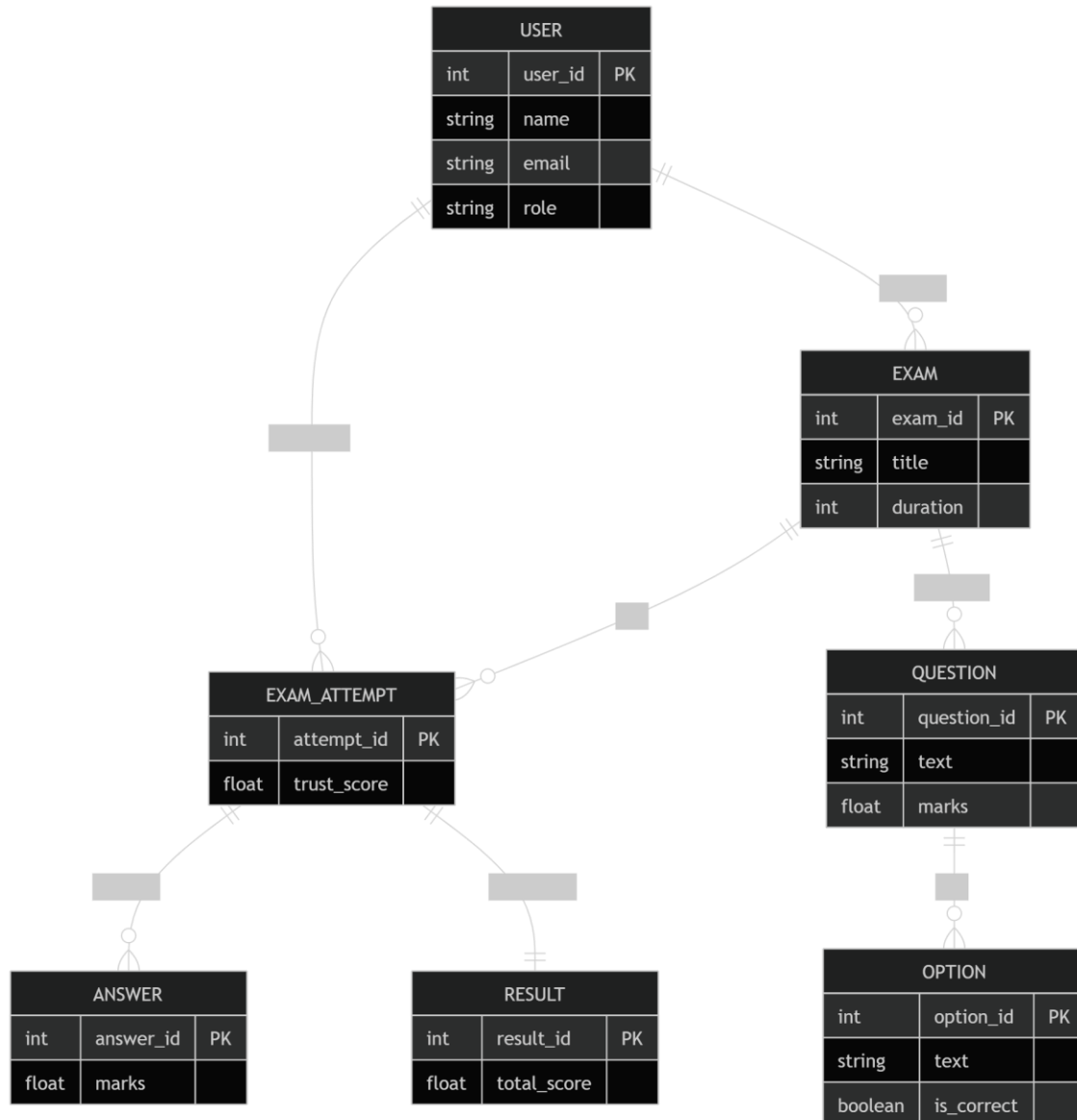
2. Application Layer → Business Logic: Service requests
3. Business Logic → Data Layer: CRUD operations
4. Data Layer → Business Logic: Query results
5. Business Logic → Application Layer: Processed data
6. Application Layer → User Interface: API responses

Key Architectural Benefits:

- Layered Separation: Clear boundaries between concerns
- Scalability: Each layer can be scaled independently
- Maintainability: Changes isolated to specific layers
- Testability: Components can be unit tested in isolation
- Security: Multiple authentication and authorization checkpoints
- Component Reusability: Services can be consumed by multiple layers

13. Entity Relationship Diagram

The Entity Relationship (ER) Diagram shows the logical data model and relationships between all entities in the TrustExam database system.



Description - Entity Relationships and Data Model:

Entities and Their Relationships:

1. USER Entity (Strong Entity)

- Attributes:

- `user_id` (PK): Primary key
- `name`: User's full name

- email: Unique email address
- role: User's role in system (Student/Faculty/Admin)
- Role in System:
 - Students: Take exams and view results
 - Faculty: Create exams and review results
 - Admins: Manage system configuration
- Relationships: Creates EXAM, Participates in EXAM_ATTEMPT

2. EXAM Entity

- Attributes:
 - exam_id (PK): Primary key
 - title: Examination title
 - duration: Exam duration in minutes
- Relationships:
 - Created by USER (Many-to-One): Foreign key created_by
 - Contains QUESTION (One-to-Many)
 - Associated with EXAM_ATTEMPT (One-to-Many)

3. QUESTION Entity

- Attributes:
 - question_id (PK): Primary key
 - question_text: Complete question content
 - marks: Points allocated for question
- Relationships:
 - Belongs to EXAM (Many-to-One): Foreign key exam_id
 - Has multiple OPTION entries (One-to-Many)
 - Referenced by ANSWER (One-to-Many)

4. OPTION Entity (from OPTION_TABLE)

- Attributes:
 - option_id (PK): Primary key
 - option_text: Text of the choice
 - is_correct: Boolean flag for correct answer
- Relationships:
 - Belongs to QUESTION (Many-to-One): Foreign key question_id
 - Selected in ANSWER entries (One-to-Many)

5. EXAM_ATTEMPT Entity (Weak Entity)

- Attributes:
 - attempt_id (PK): Primary key
 - trust_score: Behavioral trust score
 - [start_time, submit_time]: Session timestamps
- Relationships:

- Student USER and EXAM (Many-to-One for each)
- Contains multiple ANSWER entries (One-to-Many)
- Produces one RESULT (One-to-One)

6. ANSWER Entity (Weak Entity)

- Attributes:
 - answer_id (PK): Primary key
 - marks: Marks obtained for this answer
- Relationships:
 - Part of EXAM_ATTEMPT (Many-to-One)
 - References QUESTION (Many-to-One)
 - Selects OPTION (Many-to-One)

7. RESULT Entity

- Attributes:
 - result_id (PK): Primary key
 - total_score: Final combined score
 - final_status: Result status (Pass/Fail/Review)
- Relationships:
 - One-to-One with EXAM_ATTEMPT: Unique foreign key

Cardinality and Participation:

- USER (1) ---creates--- (M) EXAM: Each user can create many exams
- USER (1) ---attempts--- (M) EXAM_ATTEMPT: Each user can attempt many exams
- EXAM (1) ---contains--- (M) QUESTION: Each exam has multiple questions
- QUESTION (1) ---has--- (M) OPTION_TABLE: Each question has multiple options
- EXAM_ATTEMPT (1) ---includes--- (M) ANSWER: Each attempt has multiple answers
- EXAM_ATTEMPT (1) ---generates--- (1) RESULT: Each attempt produces one result

Constraints:

- Primary Keys: Uniquely identify each entity instance
- Foreign Keys: Maintain referential integrity
- Unique Constraints: email in USER table
- Not Null Constraints: Essential data fields are mandatory
- Boolean Constraints: is_correct field restricted to 0/1

Data Integrity Rules:

- Referential Integrity: No orphaned records (answered questions must have valid question_id)
- Domain Integrity: Data types and value ranges enforced
- Key Integrity: Primary and foreign keys validated
- Entity Integrity: No duplicate entity instances

Normalization Level: Third Normal Form (3NF)

- No partial dependencies (2NF achieved)
- No transitive dependencies (3NF achieved)
- Minimal data redundancy
- Efficient query performance

14. Conclusion

The comprehensive UML diagrams and database schema documentation provided in this document present a complete overview of the TrustExam system's architecture, design, operations, and data structure.

Key System Components:

1. **Architecture:** The system follows a layered architecture with clear separation of concerns, enabling scalability and maintainability across Presentation, Application, Business Logic, and Data layers.
2. **Security:** Multiple layers of security including authentication, role-based access control, activity logging, and behavioral monitoring ensure system integrity.
3. **Monitoring:** Real-time proctoring capabilities through continuous behavioral analysis and trust score calculation detect and flag suspicious examination activities.
4. **Database Design:** Well-normalized schema (3NF) with strong referential integrity ensures data consistency and optimal query performance.
5. **Scalability:** Distributed deployment architecture and layered design support horizontal scaling and high availability.
6. **Reliability:** Comprehensive error handling, validation at multiple layers, and state management ensure reliable operation under varying load conditions.
7. **User Experience:** Multi-actor support (Students, Faculty, Administrators) with role-specific interfaces and workflows ensure intuitive user interactions.
8. **Data Integrity:** Activity logging, audit trails, comprehensive result generation with trust scoring, and referential constraints maintain data accuracy and consistency.
9. **Deployment:** The system architecture supports deployment across multiple servers with load balancing, caching mechanisms, and database replication for high availability.

10. Maintainability: Clear separation of concerns, well-defined interfaces between layers, and comprehensive documentation enable easy maintenance and future enhancements.

Implementation Roadmap:

Phase 1: Database Setup

- Create all tables with proper relationships
- Set up indexing for performance optimization
- Configure backup and recovery procedures

Phase 2: Backend Development

- Implement Business Logic Layer (Exam Engine, Trust Engine, Result Evaluation)
- Develop Application Layer APIs
- Integrate authentication and authorization

Phase 3: Frontend Development

- Build user interfaces for Student, Faculty, and Admin roles
- Implement real-time monitoring and event capture
- Design responsive dashboard layouts

Phase 4: Integration & Testing

- Integration testing across all layers
- Performance testing and optimization
- Security testing and vulnerability assessment

Phase 5: Deployment

- Set up production infrastructure
- Deploy to cloud/on-premise servers
- Establish monitoring and alerting systems

This documentation serves as:

- Reference material for development teams
- Communication tool for stakeholders
- Foundation for system testing and validation
- Guide for future system enhancements and scalability improvements

The TrustExam system, built on this architecture and schema, provides a robust, secure, and scalable platform for conducting proctored online examinations with intelligent behavioral analysis and trust-based result validation.